Facultad:

Hábitat, Infraestructura y Creatividad

Carrera:

Ingeniería en Sistemas de Información

Asignatura:

Desarrollo de Software Seguro

Tema:

Práctica de Laboratorio: Gestión de Secretos y Configuración Segura

Estudiante:

Ángel Loor

Docente:

Edison Guaña

Quito, 5 de Noviembre del 2026

1. Introducción

El manejo inadecuado de credenciales y secretos dentro del código fuente es uno de los errores más frecuentes en el desarrollo de software. Muchas veces, por rapidez o descuido, se dejan contraseñas, tokens y llaves de acceso escritas directamente en el código, sin considerar el riesgo que esto representa. Si además la aplicación se conecta a la base de datos con una cuenta administrativa, el robo de una sola credencial entrega el sistema completo.

Esta práctica de laboratorio tuvo como objetivo construir una aplicación web funcional, identificar de manera controlada sus vulnerabilidades de seguridad, corregirlas y documentar cada paso con evidencia. El trabajo se desarrolló en cuatro fases: investigación y selección de herramientas, implementación con errores intencionales, corrección segura y endurecimiento, y validación con pruebas.

La aplicación desarrollada es un inventario llamado Inventario Lab, una API con interfaz web que lista productos y usuarios y permite registrar productos mediante un token. Sirvió como escenario para demostrar vulnerabilidades como credenciales expuestas, token en el código, datos sensibles en las respuestas, endpoints de depuración y un usuario de base de datos con privilegios de administrador.

2. Fase 1 - Investigación Inicial

2.1 Aplicación desarrollada

Se desarrolló una API con interfaz web, en dos versiones, con las siguientes funcionalidades:

- Consulta de productos del inventario
- Consulta de usuarios (nombre y correo)
- Registro de productos protegido con un token de API
- Versión insegura con endpoints de depuración (/debug) y de SQL libre (/admin/sql)
- Versión segura con verificaciones en vivo de sus propias protecciones

2.2 Herramientas y tecnologías seleccionadas

Para el desarrollo del proyecto se utilizaron las siguientes herramientas:

Herramienta	Uso en el proyecto
Visual Studio Code	Editor de código principal para desarrollar la aplicación
Node.js v24.11.1	Entorno de ejecución para el servidor backend

Express.js	Framework web para manejar rutas y archivos estáticos
PostgreSQL 18	Motor de base de datos relacional para almacenar productos y usuarios
node-postgres (pg)	Controlador de PostgreSQL con consultas parametrizadas
dotenv	Carga de variables de entorno desde archivos .env por ambiente
Git y GitHub	Control de versiones y repositorio remoto del proyecto
Terminal y Google Chrome	Ejecución de comandos y pruebas de la aplicación en localhost
Archivo .env	Gestión segura de variables de entorno (Fase 3)

Como métodos seguros de gestión de secretos se investigaron: variables de entorno (usado en esta práctica), archivos .env excluidos del repositorio, gestores de secretos dedicados (de un proveedor de nube o HashiCorp Vault) y secretos cifrados del sistema de integración continua. Los dos últimos son recomendables para producción porque añaden rotación y auditoría.

2.3 Estructura del proyecto

El proyecto se organizó en carpetas separadas para distinguir claramente la versión insegura de la versión corregida:

```
lab-secretos/
  vulnerable/   <- Servidor con vulnerabilidades intencionales (app.js, index.html)
  seguro/       <- Versión corregida (app.js, config.js, public/)
  db/           <- Scripts SQL (rol inseguro y esquema con mínimo privilegio)
  scripts/      <- setup-db.js (roles y .env) y scan-secrets.js (escáner)
  tests/        <- pruebas.js (28 pruebas funcionales y de seguridad)
  ui/           <- Estilos compartidos
  .env.*        <- Variables de entorno protegidas (no versionadas)
  .gitignore    <- Exclusión de archivos sensibles
```

2.4 Diseño de la base de datos

Se crearon bases de datos en PostgreSQL con un diseño sencillo y las siguientes tablas:

- usuarios: almacena id, nombre, email y password_hash
- productos: almacena id, nombre y precio (con restricción de precio mayor o igual a cero)

La versión insegura usa la base lab_vuln con el rol lab_vuln_admin (superusuario). La versión segura usa dos bases separadas, lab_dev y lab_prod, cada una con un rol dueño para el DDL (lab_owner_*) y un rol de aplicación de mínimo privilegio (lab_app_*).

3. Fase 2 - Identificación de Vulnerabilidades

En esta fase se configuró la aplicación de forma intencionalmente insegura para demostrar los riesgos reales de un desarrollo sin buenas prácticas. Todas las credenciales son ficticias y la aplicación corre en http://localhost:3001. A continuación se describe cada vulnerabilidad identificada e implementada.

3.1 Resumen de vulnerabilidades encontradas

Vulnerabilidad	Ubicación en el código	Riesgo
Credenciales hardcodeadas	vulnerable/app.js, líneas 9-16	Cualquiera con acceso al código ve la contraseña de la base de datos
Token de API expuesto	vulnerable/app.js, línea 20	Token imposible de rotar y visible para todos
Usuario superadmin en BD	db/00_vulnerable.sql, línea 2	La app tiene permisos ilimitados en la base de datos
Datos sensibles sin protección	vulnerable/app.js, GET /api/usuarios	La API devuelve el password_hash de todos los usuarios
Endpoint de depuración	vulnerable/app.js, GET /debug	Revela usuario, base y token sin autenticación
SQL libre con stack trace	vulnerable/app.js, POST /admin/sql	Ejecución arbitraria como superusuario y fuga de detalles internos
Secretos en registros	vulnerable/app.js, línea 57	Token y clave quedan impresos en consola y logs

3.2 Vulnerabilidad 1 - Credenciales hardcodeadas en el código

La primera y más grave vulnerabilidad es que las credenciales de conexión a la base de datos están escritas directamente en el archivo app.js. Cualquier persona que tenga acceso al repositorio o al código fuente puede ver la contraseña real de la base de datos.

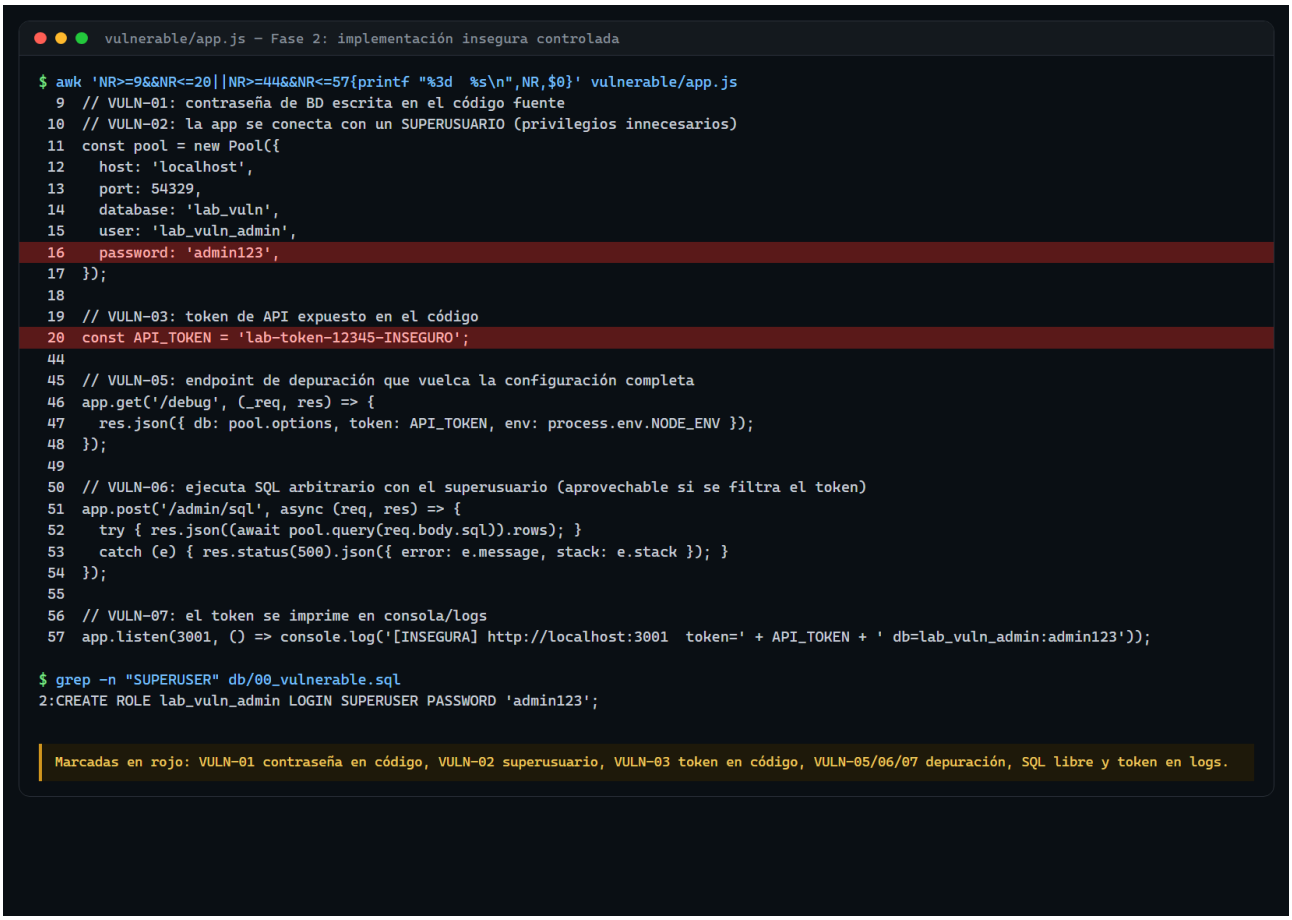


Ilustración 1 Credenciales, token y endpoints inseguros visibles en el código fuente (vulnerable/app.js)

3.3 Vulnerabilidad 2 - Token de API expuesto

El token que autoriza la escritura también está fijo en el código. Si se filtra el repositorio, un atacante puede crear registros, y para cambiarlo hay que modificar el código y volver a desplegar.

```
const API_TOKEN = 'lab-token-12345-INSEGURO';   // Valor débil, fijo y expuesto
```

3.4 Vulnerabilidad 3 - Datos sensibles sin protección

La consulta de usuarios usa SELECT *, por lo que la API devuelve también el hash de la contraseña a cualquier visitante, sin autenticación. Con el hash, un atacante puede intentar descifrarlo fuera de línea.

```
const r = await pool.query('SELECT * FROM usuarios ORDER BY id');   // incluye
password_hash
```

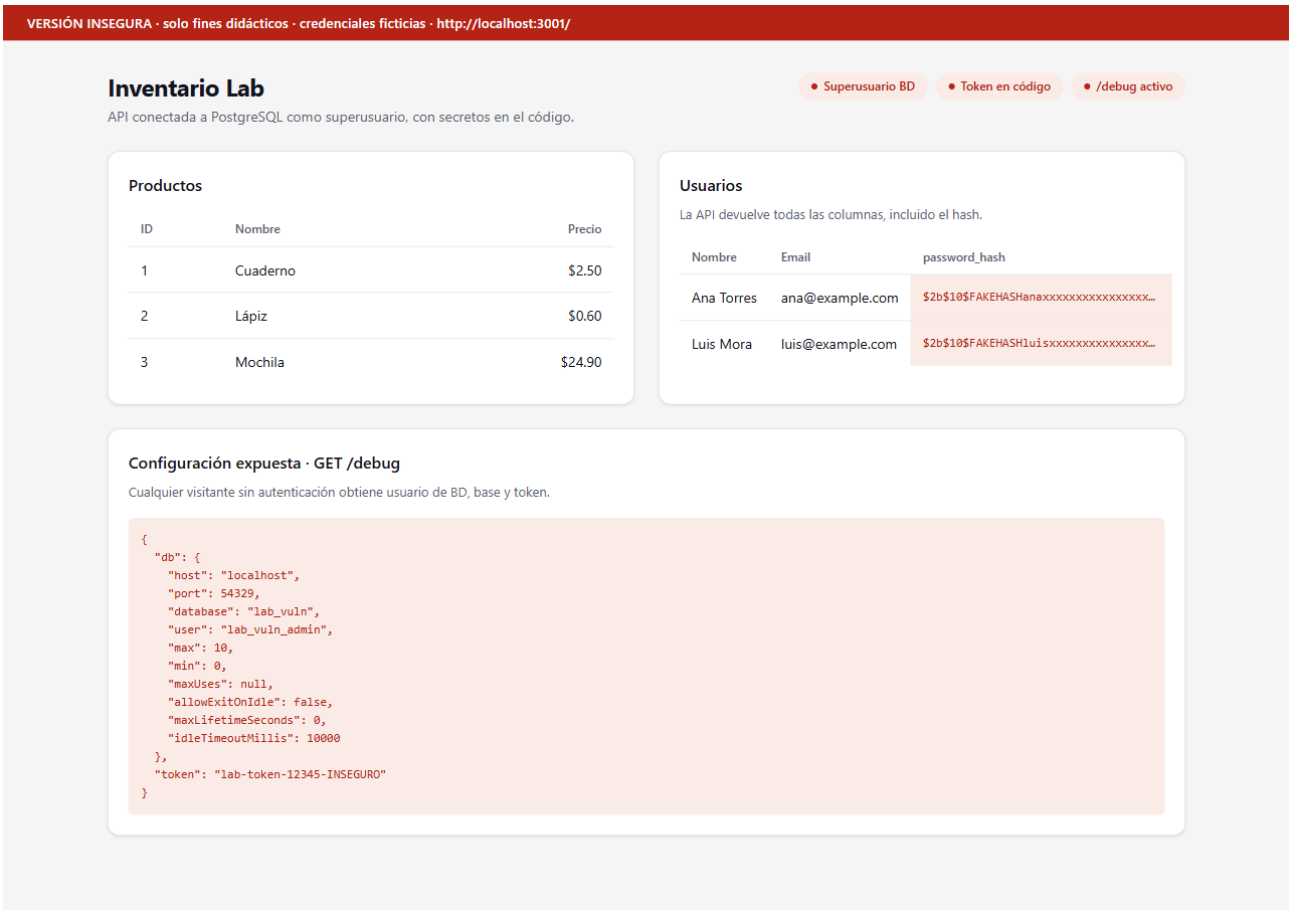


Ilustración 2 Aplicación insegura en http://localhost:3001: muestra el hash de contraseña y la configuración completa

3.5 Vulnerabilidad 4 - Endpoint de depuración y SQL libre

La ruta /debug devuelve la configuración de la conexión y el token, y la ruta /admin/sql ejecuta cualquier sentencia SQL que reciba, devolviendo además el stack trace en los errores. Combinadas con la cuenta de superusuario, permiten controlar toda la base de datos.

```
app.get('/debug', (_req, res) => {
  res.json({ db: pool.options, token: API_TOKEN, env: process.env.NODE_ENV });
});
app.post('/admin/sql', async (req, res) => {
  try { res.json((await pool.query(req.body.sql)).rows); }
  catch (e) { res.status(500).json({ error: e.message, stack: e.stack }); }
});
```

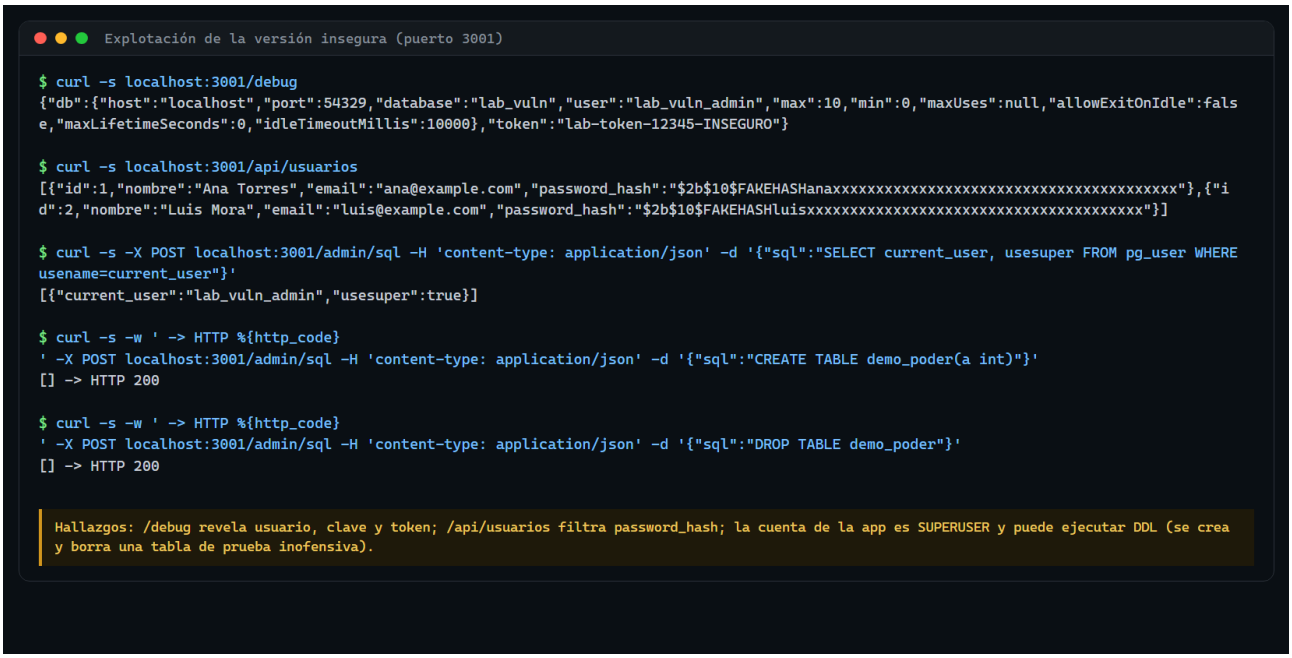


Ilustración 3 Explotación de la versión insegura: /debug, hash expuesto y DDL ejecutado con la cuenta de la aplicación

3.6 Vulnerabilidad 5 - Secretos impresos en los registros

Al arrancar, el servidor imprime el token y la clave de la base de datos en la consola. Esos mensajes suelen terminar en archivos de registro, herramientas de monitoreo y capturas de pantalla.

```
app.listen(3001, () => console.log('[INSEGURA] ... token=' + API_TOKEN + '
db=lab_vuln_admin:admin123'));
```

3.7 Vulnerabilidad 6 - Usuario con privilegios excesivos en la base de datos

La aplicación se conecta con el rol lab_vuln_admin, creado como SUPERUSER. Si alguien explota otra vulnerabilidad y toma control de la aplicación, tendrá acceso total a la base de datos: podrá crear y eliminar tablas, modificar usuarios y crear nuevos roles.

```
CREATE ROLE lab_vuln_admin LOGIN SUPERUSER PASSWORD 'admin123';
```



Ilustración 4 Escáner de secretos: 3 hallazgos en la versión insegura y 0 en la segura

4. Fase 3 - Corrección Segura y Endurecimiento

Se corrigió la aplicación en la carpeta seguro/ aplicando variables de entorno, exclusión de secretos con .gitignore, mínimo privilegio en la base de datos y separación de ambientes. La siguiente tabla relaciona cada vulnerabilidad con su solución.

Vulnerabilidad	Solución aplicada	Archivo
Credenciales y token hardcodeados	Variables de entorno cargadas desde .env.<ambiente>, generadas al azar con crypto.randomBytes	seguro/config.js, scripts/setup-db.js
Usuario superadmin	Rol lab_app_* sin superusuario, createdb ni createrole; SELECT e INSERT solo donde hace falta	db/01_esquema_seguro.sql
Datos sensibles	GRANT SELECT (id, nombre, email) por columna y consulta explícita de columnas	db/01_esquema_seguro.sql, seguro/app.js
/debug y /admin/sql	Endpoints eliminados; errores genéricos al cliente y detalle solo en el registro interno	seguro/app.js
Secretos en registros	El arranque imprime entorno y usuario de BD, nunca contraseñas ni tokens	seguro/app.js
Secretos en el repositorio	.gitignore excluye .env y .env.*; solo se versionan plantillas *.example sin valores	.gitignore
Un solo ambiente	Development y production con BD, usuario, contraseña, token y puerto distintos	scripts/setup-db.js

4.1 Archivo de variables de entorno y validación

El módulo config.js carga .env.development o .env.production según NODE_ENV y valida las siete variables requeridas. Si faltan, termina con código 1 mostrando solo sus nombres, nunca sus valores; en producción exige además un token de al menos 32 caracteres. Las contraseñas y el token se generan con el script setup-db.js, por lo que ningún secreto real existe en el código ni en este informe.

```
PORT=3000
DB_HOST=localhost
DB_NAME=lab_dev
DB_USER=lab_app_dev
DB_PASSWORD=<generada-por-setup-db>
API_TOKEN=<generado-por-setup-db>
```

4.2 Exclusión de secretos con .gitignore

Exclusión de secretos con .gitignore

```
$ cat .gitignore
# Secretos: NUNCA se versionan
.env
.env.*
!.env.example
!.env.development.example
!.env.production.example
*.pem
*.key
node_modules/
logs/
*.log

$ ls -1A | grep -E '^\.env'
.env.development
.env.development.example
.env.production
.env.production.example

$ git check-ignore -v .env.development .env.production
lab-secretos/.gitignore:3:.env.*        .env.development
lab-secretos/.gitignore:3:.env.*        .env.production

$ git add -n . | grep -i env || true
add 'lab-secretos/.env.development.example'
add 'lab-secretos/.env.production.example'

$ git ls-files -- '.env*'
```

> Los archivos .env.development y .env.production existen en disco, pero git los ignora: solo se versionan las plantillas *.example (sin valores).

Ilustración 5 Los archivos .env existen en disco pero Git los ignora; solo se versionan las plantillas

4.3 Mínimo privilegio en la base de datos

El rol dueño ejecuta el esquema y el rol de aplicación solo recibe SELECT e INSERT sobre productos y SELECT de las columnas id, nombre y email de usuarios, sin password_hash. Además se revocó el permiso de conexión a PUBLIC en cada base, de modo que cada rol solo entra a su ambiente.

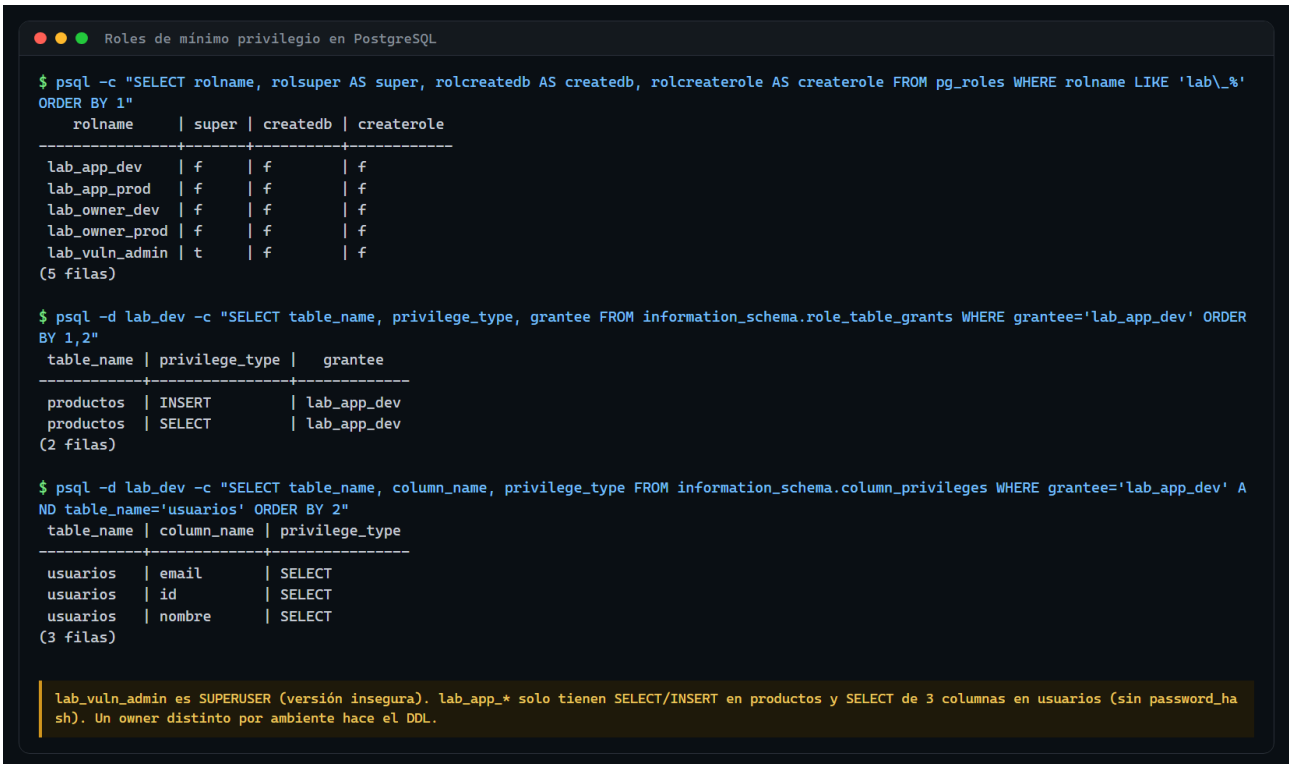


Ilustración 6 Roles sin atributos de administrador y permisos por tabla y por columna

4.4 Separación de ambientes

Elemento	Development	Production
Archivo de configuración	.env.development	.env.production
Base de datos	lab_dev	lab_prod
Rol de aplicación / de migraciones	lab_app_dev / lab_owner_dev	lab_app_prod / lab_owner_prod
Contraseña y token	Aleatorios, propios	Aleatorios, propios (token de 32 o más caracteres)
Puerto	3000	8080

5. Fase 4 - Validación, Análisis y Pruebas

La versión segura se ejecutó con npm run dev en http://localhost:3000. La página consulta la API real y muestra productos y usuarios sin password_hash. El panel «Verificaciones en vivo» realiza peticiones reales al cargar: /debug y /admin/sql responden 404 y escribir sin token responde 401.

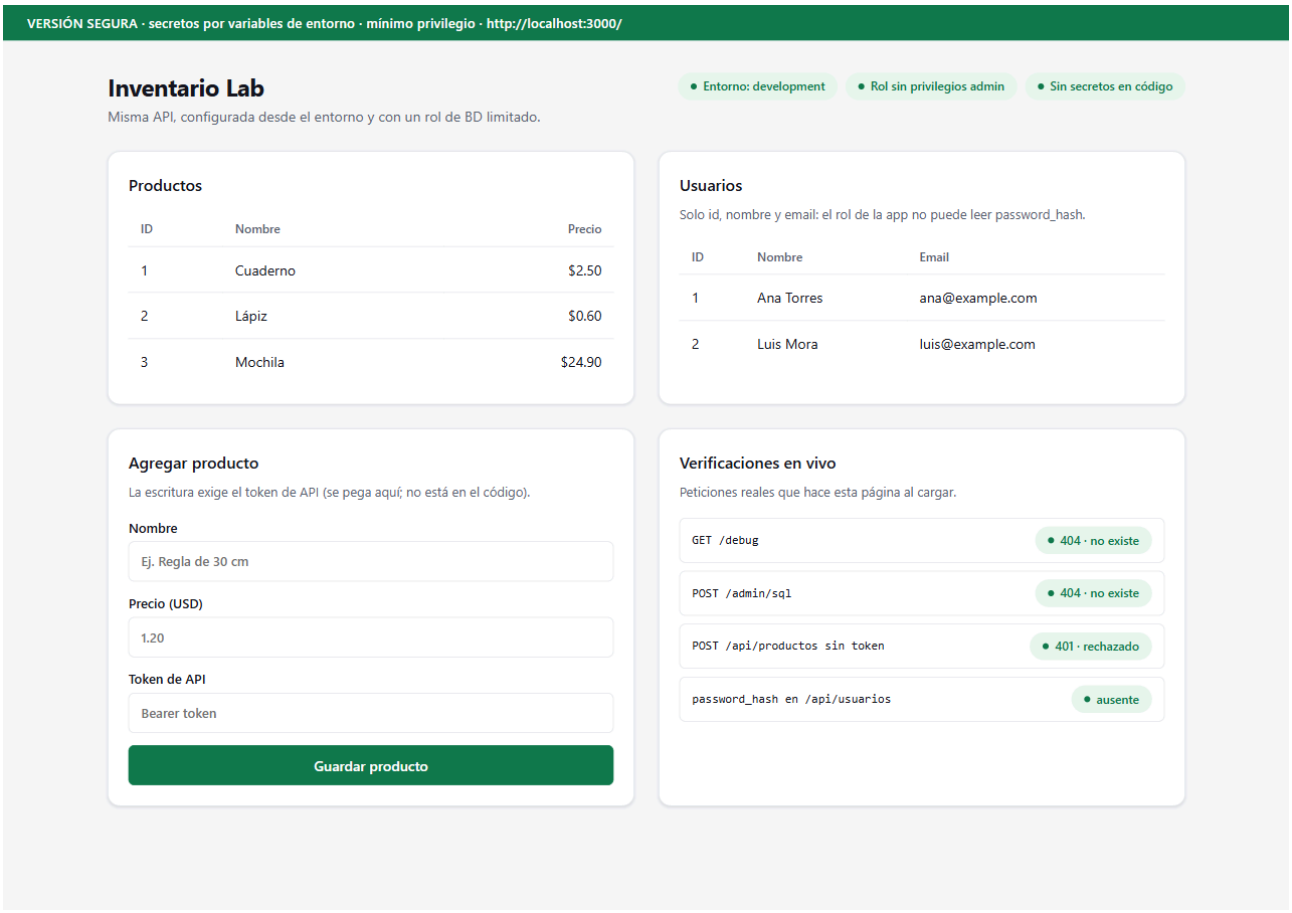


Ilustración 7 Aplicación segura funcionando en http://localhost:3000, entorno development

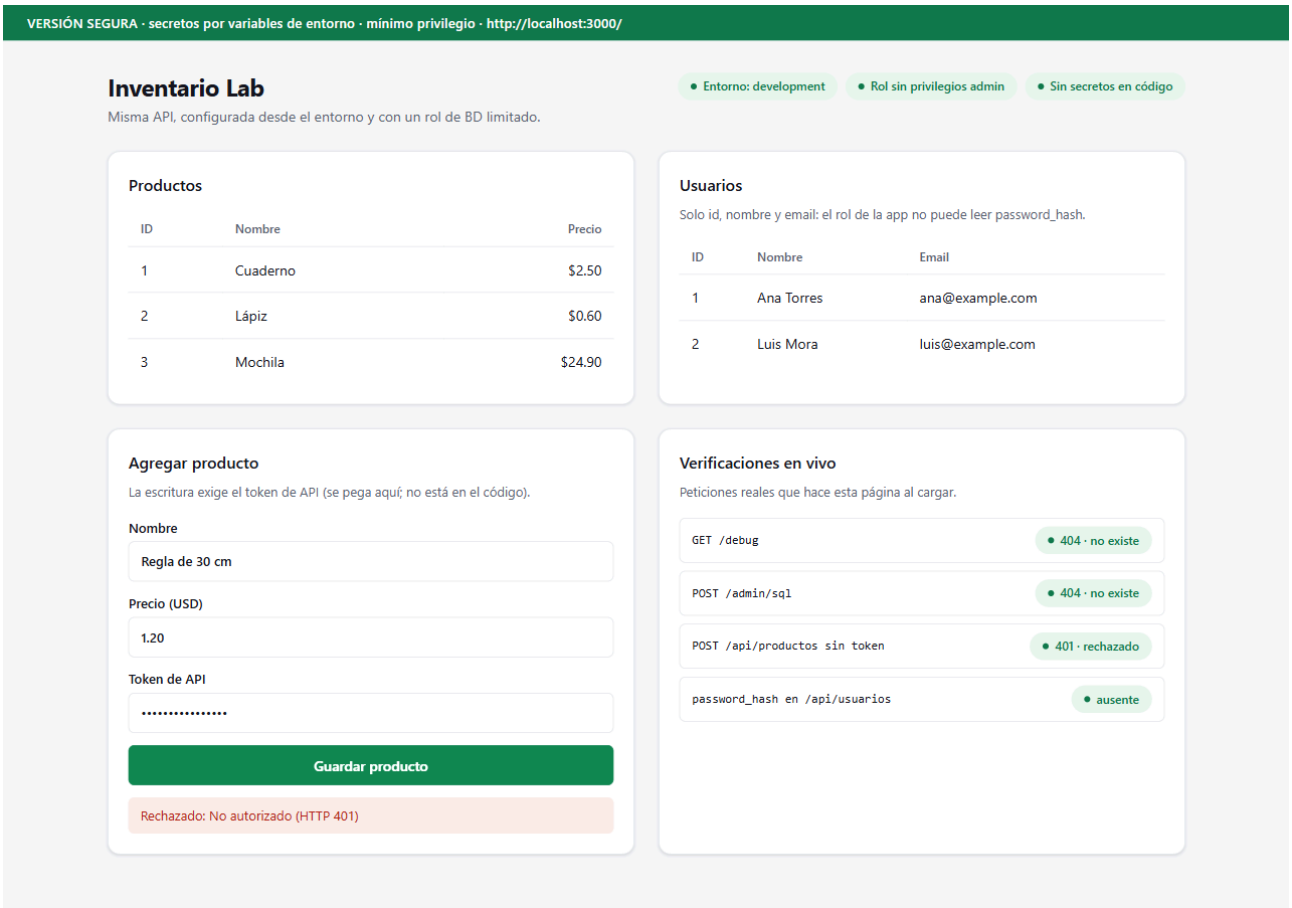


Ilustración 8 Prueba funcional: guardar un producto con un token incorrecto es rechazado (HTTP 401)

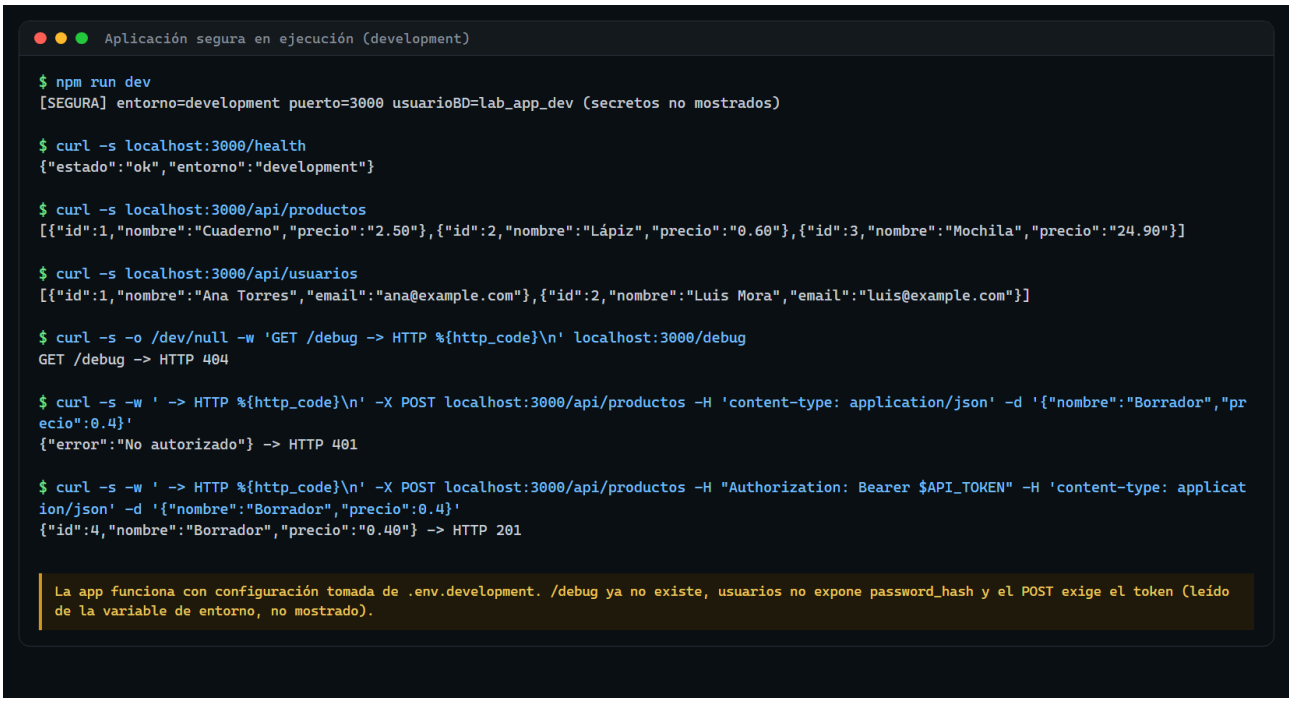


Ilustración 9 Pruebas con curl: el POST con token válido responde 201 (el token se leyó de la variable de entorno y no se muestra)

Después se simuló que un atacante roba las credenciales del rol de la aplicación. La base de datos rechazó todo salvo la lectura permitida, y el usuario de development tampoco puede conectarse a la base de production.

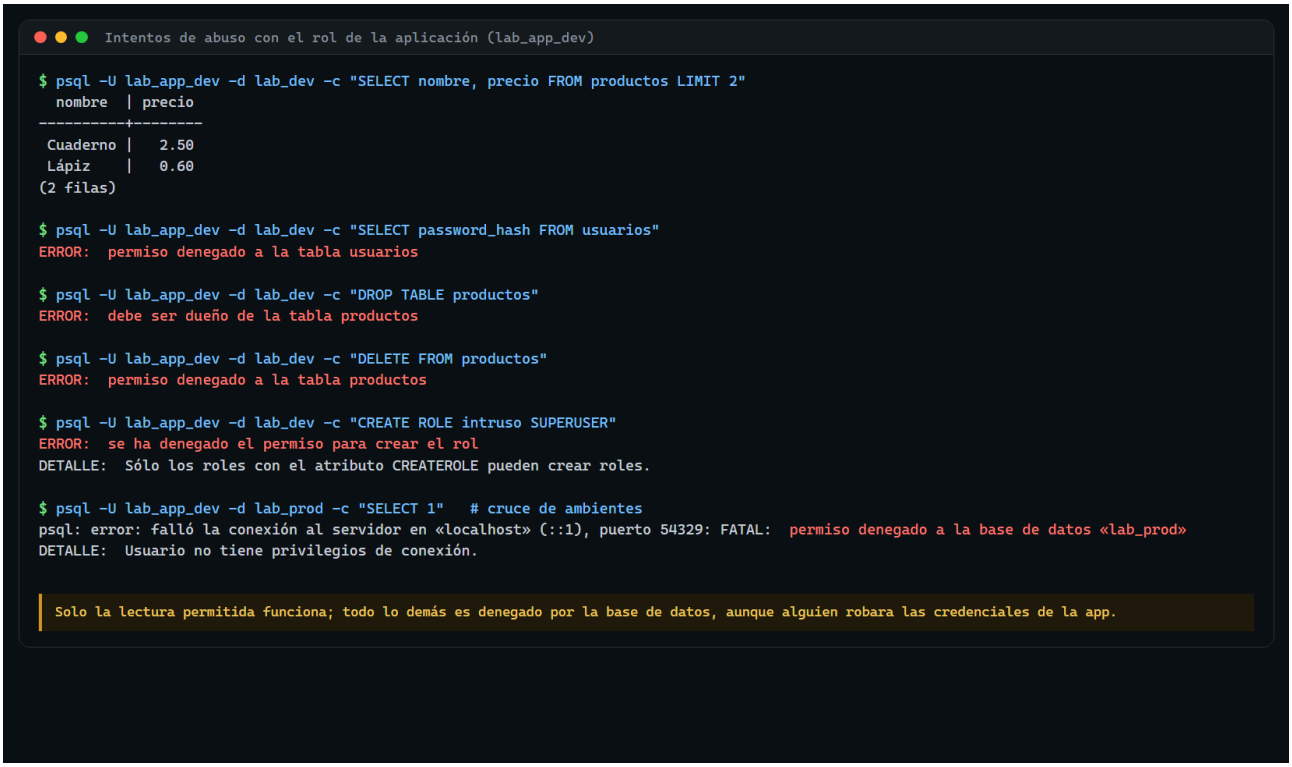


Ilustración 10 Intentos de leer password_hash, borrar tablas y crear roles: denegados por PostgreSQL

Finalmente, el script tests/pruebas.js (npm test) arranca la aplicación en ambos ambientes y ejecuta 28 comprobaciones en cinco grupos: funcionales, control de acceso, mínimo privilegio, separación

de ambientes y configuración. También verifica que ninguna respuesta ni registro contenga los secretos reales.

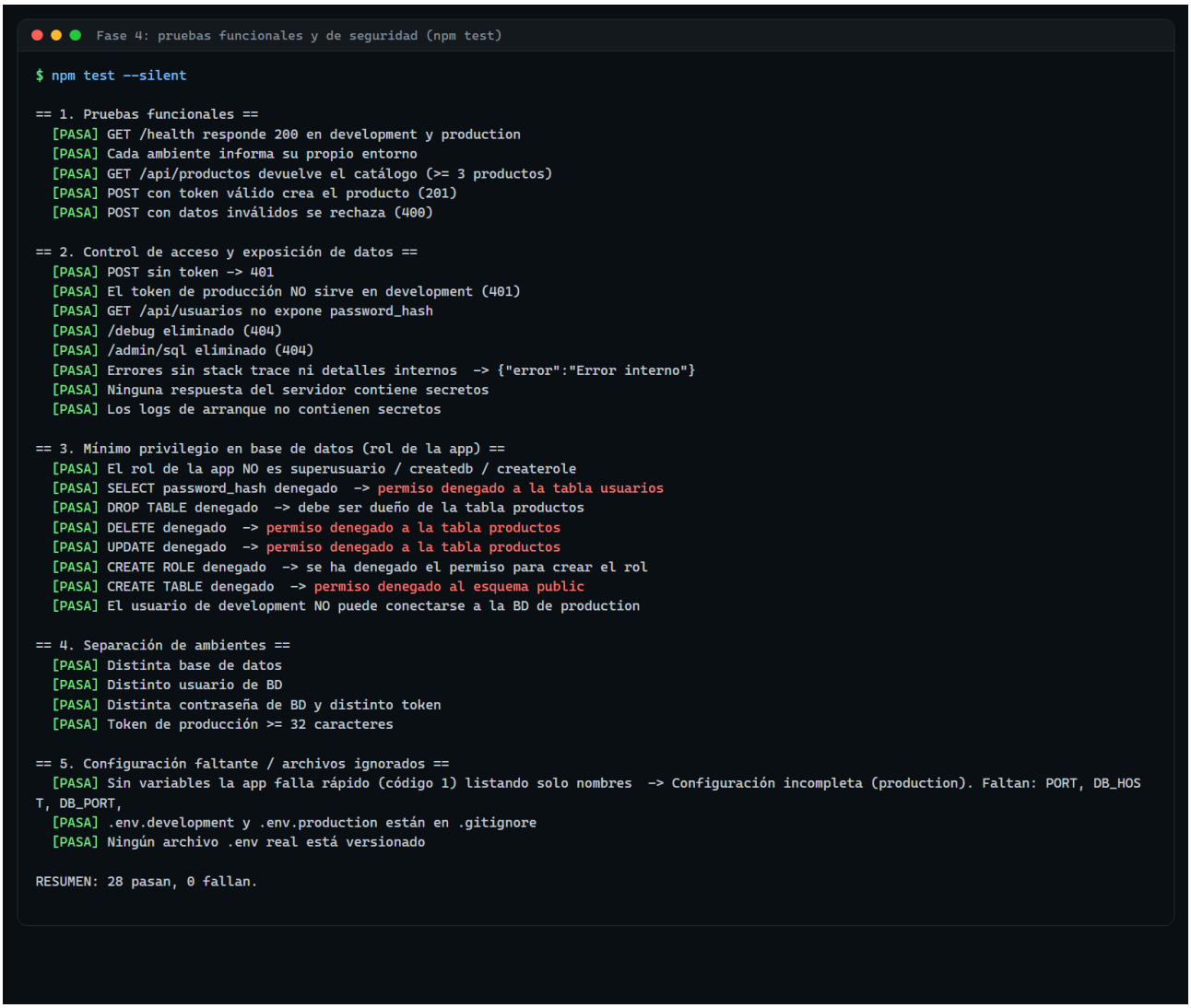


Ilustración 11 Resultado de npm test: 28 pruebas pasan, 0 fallan



Ilustración 12 Sin configuración la aplicación falla rápido sin imprimir valores; las plantillas no contienen secretos

5.1 Vulnerabilidades corregidas y riesgos mitigados

Aspecto	Versión insegura	Versión segura

Contraseña de BD	Literal en el código	Variable de entorno aleatoria
Token de API	Fijo y visible	Variable de entorno; comparación en tiempo constante
Rol de la aplicación	Superusuario	Sin privilegios de administrador
Datos sensibles	password_hash expuesto	Sin acceso (a nivel de BD y de API)
Depuración	/debug y /admin/sql activos	Eliminados (404)
Registros de arranque	Imprime token y clave	Sin secretos
Ambientes	Uno solo	Dos, aislados
Hallazgos del escáner	3	0

Limitaciones. Los archivos .env siguen existiendo en el disco del equipo; en producción conviene un gestor de secretos con rotación y auditoría. Si un secreto se hubiera confirmado en Git antes de aplicar .gitignore, seguiría en el historial y habría que rotarlo. Tampoco se configuró TLS hacia la base de datos ni limitación de peticiones, y las pruebas se hicieron en un entorno local de laboratorio.

6. Conclusiones

- Esta práctica demostró que las vulnerabilidades más peligrosas no son las más complejas. Dejar una contraseña en el código o conectar la aplicación con un superusuario puede comprometer por completo un sistema, y corregirlo requirió pocas líneas.
- La seguridad debe ser parte del desarrollo desde el inicio, no un paso opcional al final. Aplicar variables de entorno, .gitignore y configuración validada no requiere un esfuerzo adicional significativo, pero la diferencia en protección es enorme.
- Configurar correctamente los permisos de la base de datos es tan importante como proteger el código: con el rol limitado, aun con las credenciales robadas, PostgreSQL rechazó leer password_hash, borrar tablas o crear roles.
- Separar ambientes con credenciales distintas evita que una filtración en desarrollo comprometa producción.

7. Recomendaciones

Basado en las vulnerabilidades identificadas, se recomienda:

- Usar variables de entorno: las credenciales nunca deben estar en el código; deben almacenarse en archivos .env excluidos del repositorio con .gitignore, y en producción en un gestor de secretos con rotación.
- Aplicar mínimo privilegio: una cuenta distinta para migraciones y para la aplicación, con permisos por tabla y por columna.
- Automatizar la detección: ejecutar el escáner de secretos como pre-commit hook y en la integración continua.
- Rotar de inmediato cualquier secreto que llegue a confirmarse en Git, ya que .gitignore no borra el historial.
- Activar TLS hacia la base de datos y reemplazar el token estático por autenticación con expiración.

8. Referencias

- OWASP Foundation. (2021). A05:2021 Security Misconfiguration. https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- OWASP Foundation. Secrets Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- MITRE. CWE-798: Use of Hard-coded Credentials. https://cwe.mitre.org/data/definitions/798.html
- MITRE. CWE-532: Insertion of Sensitive Information into Log File. https://cwe.mitre.org/data/definitions/532.html
- The Twelve-Factor App. III. Config. https://12factor.net/config
- PostgreSQL Global Development Group. PostgreSQL Documentation: Privileges. https://www.postgresql.org/docs/current/ddl-priv.html
- Express.js. Express - Node.js web application framework. https://expressjs.com/
- dotenv (motdotla). https://github.com/motdotla/dotenv

Anexo. Código corregido

lab-secretos/vulnerable/app.js

```
// ============================================================
// VERSIÓN INSEGURA (Fase 2) — SOLO PARA FINES DIDÁCTICOS
// No usar en ningún entorno real. Todas las credenciales son ficticias.
// ============================================================
const path = require('path');
```

```
const express = require('express');
const { Pool } = require('pg');

// VULN-01: contraseña de BD escrita en el código fuente
// VULN-02: la app se conecta con un SUPERUSUARIO (privilegios innecesarios)
const pool = new Pool({
  host: 'localhost',
  port: 54329,
  database: 'lab_vuln',
  user: 'lab_vuln_admin',
  password: 'admin123',
});

// VULN-03: token de API expuesto en el código
const API_TOKEN = 'lab-token-12345-INSEGURO';

const app = express();
app.use(express.json());
app.use('/ui', express.static(path.join(__dirname, '..', 'ui')));
app.get('/', (_req, res) => res.sendFile(path.join(__dirname, 'index.html')));

app.get('/api/productos', async (_req, res) => {
  const r = await pool.query('SELECT * FROM productos ORDER BY id');
  res.json(r.rows);
});

// VULN-04: SELECT * devuelve también password_hash
app.get('/api/usuarios', async (_req, res) => {
  const r = await pool.query('SELECT * FROM usuarios ORDER BY id');
  res.json(r.rows);
});

app.post('/api/productos', async (req, res) => {
  if (req.headers.authorization !== 'Bearer ' + API_TOKEN) return res.sendStatus(401);
  const { nombre, precio } = req.body;
  const r = await pool.query('INSERT INTO productos (nombre,precio) VALUES ($1,$2) RETURNING *', [nombre, preci
o]);
  res.status(201).json(r.rows[0]);
});

// VULN-05: endpoint de depuración que vuelca la configuración completa
app.get('/debug', (_req, res) => {
  res.json({ db: pool.options, token: API_TOKEN, env: process.env.NODE_ENV });
});

// VULN-06: ejecuta SQL arbitrario con el superusuario (aprovechable si se filtra el token)
app.post('/admin/sql', async (req, res) => {
  try { res.json((await pool.query(req.body.sql)).rows); }
  catch (e) { res.status(500).json({ error: e.message, stack: e.stack }); }
});

// VULN-07: el token se imprime en consola/logs
app.listen(3001, () => console.log('[INSEGURA] http://localhost:3001  token=' + API_TOKEN + ' db=lab_vuln_admi
n:admin123'));
```

lab-secretos/seguro/config.js

```
// Carga de configuración segura: secretos SOLO desde variables de entorno / archivo .env ignorado por git.
const path = require('path');
const dotenv = require('dotenv');

const entorno = process.env.NODE_ENV === 'production' ? 'production' : 'development';
// Separación de ambientes: .env.development vs .env.production (distintos usuarios, BD y tokens).
// dotenv no pisa variables ya definidas, así que en producción real mandan las del gestor/CI.
dotenv.config({ path: process.env.DOTENV_PATH || path.join(__dirname, '..', `.env.${entorno}`) });
```

```
const REQUERIDAS = ['PORT', 'DB_HOST', 'DB_PORT', 'DB_NAME', 'DB_USER', 'DB_PASSWORD', 'API_TOKEN'];
const faltan = REQUERIDAS.filter((k) => !process.env[k]);
if (faltan.length) {
  // Falla rápido, sin imprimir valores: solo los nombres de las variables que faltan.
  console.error(`Configuración incompleta (${entorno}). Faltan: ${faltan.join(', ')}`);
  process.exit(1);
}
if (entorno === 'production' && process.env.API_TOKEN.length < 32) {
  console.error('API_TOKEN demasiado corto para producción (mínimo 32 caracteres).');
  process.exit(1);
}

module.exports = Object.freeze({
  entorno,
  port: Number(process.env.PORT),
  db: Object.freeze({
    host: process.env.DB_HOST, port: Number(process.env.DB_PORT),
    database: process.env.DB_NAME, user: process.env.DB_USER, password: process.env.DB_PASSWORD,
  }),
  apiToken: process.env.API_TOKEN,
});
```

lab-secretos/seguro/app.js

```
// VERSIÓN SEGURA (Fase 3): secretos por entorno, mínimo privilegio, sin fugas de información.
const crypto = require('crypto');
const path = require('path');
const express = require('express');
const { Pool } = require('pg');
const cfg = require('./config');

const pool = new Pool(cfg.db);
const app = express();
app.disable('x-powered-by');
app.use(express.json({ limit: '10kb' }));
app.use('/ui', express.static(path.join(__dirname, '..', 'ui')));
app.use(express.static(path.join(__dirname, 'public')));

// Comparación en tiempo constante del token (evita ataques de temporización)
function tokenValido(cabecera = '') {
  const dado = Buffer.from(cabecera.replace(/^Bearer /, ''));
  const esperado = Buffer.from(cfg.apiToken);
  return dado.length === esperado.length && crypto.timingSafeEqual(dado, esperado);
}
const exigirToken = (req, res, next) =>
  (tokenValido(req.headers.authorization) ? next() : res.status(401).json({ error: 'No autorizado' }));

app.get('/health', (_req, res) => res.json({ estado: 'ok', entorno: cfg.entorno }));

app.get('/api/productos', async (_req, res, next) => {
  try { res.json((await pool.query('SELECT id, nombre, precio FROM productos ORDER BY id')).rows); } catch (e)
{ next(e); }
});

// Solo columnas permitidas por el GRANT (sin password_hash)
app.get('/api/usuarios', async (_req, res, next) => {
  try { res.json((await pool.query('SELECT id, nombre, email FROM usuarios ORDER BY id')).rows); } catch (e) {
next(e); }
});

app.post('/api/productos', exigirToken, async (req, res, next) => {
  const { nombre, precio } = req.body || {};
  if (typeof nombre !== 'string' || !nombre.trim() || nombre.length > 80 || !(Number(precio) >= 0)) {
    return res.status(400).json({ error: 'Datos inválidos' });
  }
  try {
```

```
    const r = await pool.query(
      'INSERT INTO productos (nombre, precio) VALUES ($1,$2) RETURNING id, nombre, precio', [nombre.trim(), pre
cio]);
    res.status(201).json(r.rows[0]);
  } catch (e) { next(e); }
});

// Los endpoints /debug y /admin/sql de la versión insegura se ELIMINARON.

// Errores genéricos: el detalle va a logs internos, nunca al cliente ni con secretos
app.use((err, _req, res, _next) => {
  console.error(`[error] ${err.code || ''} ${err.message}`);
  res.status(500).json({ error: 'Error interno' });
});

app.listen(cfg.port, () =>
  console.log(`[SEGURA] entorno=${cfg.entorno} puerto=${cfg.port} usuarioBD=${cfg.db.user} (secretos no mostrad
os)`));
```

lab-secretos/scripts/setup-db.js

```
// Crea BD, roles de mínimo privilegio y archivos .env.<entorno> con secretos ALEATORIOS.
// Las credenciales de administración se leen de variables de entorno (nunca del código):
//   PG_ADMIN_HOST, PG_ADMIN_PORT, PG_ADMIN_USER, PG_ADMIN_PASSWORD
const crypto = require('crypto');
const fs = require('fs');
const path = require('path');
const { Client } = require('pg');

const admin = {
  host: process.env.PG_ADMIN_HOST || 'localhost',
  port: Number(process.env.PG_ADMIN_PORT || 5432),
  user: process.env.PG_ADMIN_USER || 'postgres',
  password: process.env.PG_ADMIN_PASSWORD,
};
if (!admin.password) { console.error('Define PG_ADMIN_PASSWORD en el entorno.'); process.exit(1); }

const ENTORNOS = [
  { nombre: 'development', sufijo: 'dev', puerto: 3000 },
  { nombre: 'production', sufijo: 'prod', puerto: 8080 },
];
const rand = (n) => crypto.randomBytes(n).toString('hex');
const ident = (s) => '"' + s.replace(/"/g, '""') + '"';
const lit = (s) => "'" + s.replace(/'/g, "''") + "'";

async function conectar(database) {
  const c = new Client({ ...admin, database });
  await c.connect();
  return c;
}

(async () => {
  const schema = fs.readFileSync(path.join(__dirname, '..', 'db', '01_esquema_seguro.sql'), 'utf8');
  for (const e of ENTORNOS) {
    const owner = `lab_owner_${e.sufijo}`, app = `lab_app_${e.sufijo}`, db = `lab_${e.sufijo}`;
    const pwOwner = rand(16), pwApp = rand(16), token = rand(24); // 48 caracteres hex

    const c = await conectar('postgres');
    for (const [rol, pw] of [[owner, pwOwner], [app, pwApp]]) {
      const existe = (await c.query('SELECT 1 FROM pg_roles WHERE rolname=$1', [rol])).rowCount;
      // Roles SIN superusuario, sin CREATEDB, sin CREATEROLE
      await c.query(`${existe ? 'ALTER' : 'CREATE'} ROLE ${ident(rol)} LOGIN NOSUPERUSER NOCREATEDB NOCREATEROL
E PASSWORD ${lit(pw)}`);
    }
    await c.query(`DROP DATABASE IF EXISTS ${ident(db)}`);
    await c.query(`CREATE DATABASE ${ident(db)} OWNER ${ident(owner)}`);
```

```
    await c.query(`REVOKE CONNECT ON DATABASE ${ident(db)} FROM PUBLIC`);       // aislamiento entre ambientes
    await c.query(`GRANT CONNECT ON DATABASE ${ident(db)} TO ${ident(app)}`);
    await c.end();

    // El esquema se crea con el rol OWNER (DDL); la app solo recibe los GRANT mínimos
    const o = new Client({ ...admin, database: db, user: owner, password: pwOwner });
    await o.connect();
    await o.query(schema.replace(/:"app"/g, ident(app)));
    await o.end();

    const archivo = path.join(__dirname, '..', `.env.${e.nombre}`);
    fs.writeFileSync(archivo, [
      `NODE_ENV=${e.nombre}`, `PORT=${e.puerto}`, `DB_HOST=${admin.host}`, `DB_PORT=${admin.port}`,
      `DB_NAME=${db}`, `DB_USER=${app}`, `DB_PASSWORD=${pwApp}`, `API_TOKEN=${token}`, '',
    ].join('\n'), { mode: 0o600 });
    console.log(`[ok] ${e.nombre}: BD=${db} rol app=${app} -> .env.${e.nombre} (secretos no mostrados)`);
  }
})().catch((err) => { console.error('Error:', err.message); process.exit(1); });
```

lab-secretos/db/01_esquema_seguro.sql

```
-- ESCENARIO SEGURO (Fase 3): esquema + mínimo privilegio. Se ejecuta como el rol OWNER (DDL).
-- Variable psql :app = rol de runtime de la aplicación (creado por scripts/setup-db.js).
CREATE TABLE usuarios (
  id serial PRIMARY KEY, nombre text NOT NULL, email text NOT NULL, password_hash text NOT NULL);
CREATE TABLE productos (
  id serial PRIMARY KEY, nombre text NOT NULL, precio numeric(10,2) NOT NULL CHECK (precio >= 0));
INSERT INTO usuarios (nombre,email,password_hash) VALUES
 ('Ana Torres','ana@example.com','$2b$10$FAKEHASHanaxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'),
 ('Luis Mora','luis@example.com','$2b$10$FAKEHASHluisxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx');
INSERT INTO productos (nombre,precio) VALUES ('Cuaderno',2.50),('Lápiz',0.60),('Mochila',24.90);

-- Mínimo privilegio para el rol de la aplicación
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO :"app";
GRANT SELECT, INSERT ON productos TO :"app";
GRANT USAGE ON SEQUENCE productos_id_seq TO :"app";
GRANT SELECT (id, nombre, email) ON usuarios TO :"app";   -- sin password_hash
```

lab-secretos/.gitignore

```
# Secretos: NUNCA se versionan
.env
.env.*
!.env.example
!.env.development.example
!.env.production.example
*.pem
*.key
node_modules/
logs/
*.log
```

lab-secretos/.env.development.example

```
# Plantilla SIN valores reales. `npm run setup:db` genera .env.development con secretos aleatorios.
NODE_ENV=development
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_NAME=lab_dev
DB_USER=lab_app_dev
```

```
DB_PASSWORD=<generada-por-setup-db>
API_TOKEN=<generado-por-setup-db>
```

lab-secretos/tests/pruebas.js

```
// Pruebas funcionales y de seguridad (Fase 4). Arranca la app segura en development y production.
// Uso: npm test      (requiere haber ejecutado npm run setup:db)
const { spawn, spawnSync } = require('child_process');
const fs = require('fs');
const path = require('path');
const dotenv = require('dotenv');
const { Client } = require('pg');

const raiz = path.join(__dirname, '..');
const envs = {
  development: dotenv.parse(fs.readFileSync(path.join(raiz, '.env.development'))),
  production: dotenv.parse(fs.readFileSync(path.join(raiz, '.env.production'))),
};
const secretos = Object.values(envs).flatMap((e) => [e.DB_PASSWORD, e.API_TOKEN]);

let ok = 0, fallos = 0;
const prueba = (nombre, cond, detalle = '') => {
  cond ? ok++ : fallos++;
  console.log(`  ${cond ? '[PASA]' : '[FALLA]'} ${nombre}${detalle ? '  -> ' + detalle : ''}`);
};
const pausa = (ms) => new Promise((r) => setTimeout(r, ms));
const vistas = []; // todo lo que ve un cliente, para buscar fugas de secretos

async function http(puerto, ruta, opts = {}) {
  const r = await fetch(`http://localhost:${puerto}${ruta}`, opts);
  const texto = await r.text();
  vistas.push(texto);
  return { status: r.status, texto, json: () => JSON.parse(texto) };
}

function lanzar(entorno) {
  const p = spawn(process.execPath, [path.join(raiz, 'seguro', 'app.js')], {
    env: { PATH: process.env.PATH, SystemRoot: process.env.SystemRoot, NODE_ENV: entorno }, cwd: raiz,
  });
  let salida = '';
  p.stdout.on('data', (d) => (salida += d)); p.stderr.on('data', (d) => (salida += d));
  return { p, salida: () => salida };
}

async function intentar(cliente, sql) {
  try { await cliente.query(sql); return { ok: true }; } catch (e) { return { ok: false, msg: e.message }; }
}

(async () => {
  const dev = lanzar('development'), prod = lanzar('production');
  await pausa(1800);
  const pd = Number(envs.development.PORT), pp = Number(envs.production.PORT);
  try {
    console.log('\n== 1. Pruebas funcionales ==');
    const hd = await http(pd, '/health'), hp = await http(pp, '/health');
    prueba('GET /health responde 200 en development y production', hd.status === 200 && hp.status === 200);
    prueba('Cada ambiente informa su propio entorno', hd.json().entorno === 'development' && hp.json().entorno
=== 'production');
    const prods = await http(pd, '/api/productos');
    prueba('GET /api/productos devuelve el catálogo (>= 3 productos)', prods.status === 200 && prods.json().len
gth >= 3);
    const t = envs.development.API_TOKEN;
    const creado = await http(pd, '/api/productos', { method: 'POST', headers: { 'content-type': 'application/j
son', authorization: 'Bearer ' + t }, body: JSON.stringify({ nombre: 'Regla', precio: 1.2 }) });
    prueba('POST con token válido crea el producto (201)', creado.status === 201 && creado.json().nombre === 'R
egla');
```

```
    const malo = await http(pd, '/api/productos', { method: 'POST', headers: { 'content-type': 'application/jso
n', authorization: 'Bearer ' + t }, body: JSON.stringify({ nombre: '', precio: -1 }) });
    prueba('POST con datos inválidos se rechaza (400)', malo.status === 400);

    console.log('\n== 2. Control de acceso y exposición de datos ==');
    const sinTok = await http(pd, '/api/productos', { method: 'POST', headers: { 'content-type': 'application/j
son' }, body: '{"nombre":"x","precio":1}' });
    prueba('POST sin token -> 401', sinTok.status === 401);
    const tokProd = await http(pd, '/api/productos', { method: 'POST', headers: { 'content-type': 'application/
json', authorization: 'Bearer ' + envs.production.API_TOKEN }, body: '{"nombre":"x","precio":1}' });
    prueba('El token de producción NO sirve en development (401)', tokProd.status === 401);
    const usr = await http(pd, '/api/usuarios');
    prueba('GET /api/usuarios no expone password_hash', usr.status === 200 && !usr.texto.includes('password_has
h') && !usr.texto.includes('FAKEHASH'));
    prueba('/debug eliminado (404)', (await http(pd, '/debug')).status === 404);
    prueba('/admin/sql eliminado (404)', (await http(pd, '/admin/sql', { method: 'POST' })).status === 404);
    const roto = await http(pd, '/api/productos', { method: 'POST', headers: { 'content-type': 'application/jso
n', authorization: 'Bearer ' + t }, body: '{"nombre":' });
    prueba('Errores sin stack trace ni detalles internos', !/at .*\.js|node_modules|password|lab_app/i.test(rot
o.texto), roto.texto.replace(/\s+/g, ' ').slice(0, 60));
    prueba('Ninguna respuesta del servidor contiene secretos', !vistas.some((v) => secretos.some((s) => v.inclu
des(s))));
    const logs = dev.salida() + prod.salida();
    prueba('Los logs de arranque no contienen secretos', !secretos.some((s) => logs.includes(s)));

    console.log('\n== 3. Mínimo privilegio en base de datos (rol de la app) ==');
    const e = envs.development;
    const cli = new Client({ host: e.DB_HOST, port: Number(e.DB_PORT), database: e.DB_NAME, user: e.DB_USER, pa
ssword: e.DB_PASSWORD });
    await cli.connect();
    const su = (await cli.query("SELECT rolsuper, rolcreatedb, rolcreaterole FROM pg_roles WHERE rolname = curr
ent_user")).rows[0];
    prueba('El rol de la app NO es superusuario / createdb / createrole', !su.rolsuper && !su.rolcreatedb && !s
u.rolcreaterole);
    for (const [nombre, sql] of [
      ['SELECT password_hash denegado', 'SELECT password_hash FROM usuarios'],
      ['DROP TABLE denegado', 'DROP TABLE productos'],
      ['DELETE denegado', 'DELETE FROM productos'],
      ['UPDATE denegado', 'UPDATE productos SET precio = 0'],
      ['CREATE ROLE denegado', 'CREATE ROLE intruso'],
      ['CREATE TABLE denegado', 'CREATE TABLE x (a int)'],
    ]) {
      const r = await intentar(cli, sql);
      prueba(nombre, !r.ok, r.msg);
    }
    await cli.end();
    const cruzado = new Client({ host: e.DB_HOST, port: Number(e.DB_PORT), database: envs.production.DB_NAME, u
ser: e.DB_USER, password: e.DB_PASSWORD });
    let cruzo = true; try { await cruzado.connect(); await cruzado.end(); } catch { cruzo = false; }
    prueba('El usuario de development NO puede conectarse a la BD de production', !cruzo);

    console.log('\n== 4. Separación de ambientes ==');
    prueba('Distinta base de datos', envs.development.DB_NAME !== envs.production.DB_NAME);
    prueba('Distinto usuario de BD', envs.development.DB_USER !== envs.production.DB_USER);
    prueba('Distinta contraseña de BD y distinto token', envs.development.DB_PASSWORD !== envs.production.DB_PA
SSWORD && envs.development.API_TOKEN !== envs.production.API_TOKEN);
    prueba('Token de producción >= 32 caracteres', envs.production.API_TOKEN.length >= 32);

    console.log('\n== 5. Configuración faltante / archivos ignorados ==');
    const sinCfg = spawnSync(process.execPath, ['-e', "require('./seguro/config')"], { env: { PATH: process.en
v.PATH, SystemRoot: process.env.SystemRoot, NODE_ENV: 'production', DOTENV_PATH: path.join(raiz, 'no-existe.en
v') }, cwd: raiz, encoding: 'utf8' });
    prueba('Sin variables la app falla rápido (código 1) listando solo nombres', sinCfg.status === 1 && /Falta
n: PORT/.test(sinCfg.stderr), sinCfg.stderr.trim().slice(0, 70));
    const ign = ['.env.development', '.env.production'].map((f) => spawnSync('git', ['check-ignore', '-q', f],
{ cwd: raiz }).status);
    prueba('.env.development y .env.production están en .gitignore', ign.every((x) => x === 0));
```

```
    const trk = spawnSync('git', ['ls-files', '--', '.env.development', '.env.production'], { cwd: raiz, encodi
ng: 'utf8' });
    prueba('Ningún archivo .env real está versionado', trk.stdout.trim() === '');
  } finally {
    dev.p.kill(); prod.p.kill();
  }
  console.log(`\nRESUMEN: ${ok} pasan, ${fallos} fallan.`);
  process.exit(fallos ? 1 : 0);
})().catch((e) => { console.error(e); process.exit(2); });
```